

LISTA DE EXERCÍCIOS – SIMULADO – CAPÍTULO 1, 2, 3

Prezado(a) estudante, realize esta lista de exercícios com tranquilidade e atenção aos detalhes, elabore um repositório no GitHub para responder essa lista de exercício e as demais listas da disciplina. Nesse repositório crie uma pasta para cada lista. Para as respostas das questões escritas objetivas e abertas crie um documento de texto com as respostas, já para as questões de código elabore um arquivo .c para cada questão de código. Bom trabalho!

PARTE I: QUESTÕES TEÓRICAS E ANALÍTICAS (CONCEITOS E PRECEDÊNCIA)

Questão 01. Sensibilidade a Caixa (Case Sensitivity) e Identificadores em C (Cap. 1) — A linguagem C diferencia rigorosamente letras maiúsculas e minúsculas na formação de nomes de identificadores e palavras-chave. Com base nessa premissa, analise os pares de identificadores abaixo e assinale a alternativa correta:

- a) Os nomes de variáveis 'numero' e 'Numero' referenciam o mesmo endereço de memória.
- b) A palavra-chave 'Main' com 'M' maiúsculo é reconhecida pelo compilador como ponto de entrada válido.
- c) Todos os pares de nomes ('valor'/'VALOR', 'peso'/'Peso', 'taxa'/'TAXA') representam identificadores totalmente distintos para o compilador.
- d) A sensibilidade a caixa baixa/alta depende exclusivamente do sistema operacional utilizado na compilação.

Questão 02. Especificadores de Formato, Sequências de Escape e Erros de Compilação (Cap. 1) — Um estudante iniciante escreveu o código C abaixo tentando imprimir mensagens formatadas com quebras de linha e tabulações, mas enfrentou erros de compilação. Identifique os três erros sintáticos/estruturais presentes no código:

```
#include <stdio.h>
#include <stdlib.h>;

int Main()
{
    int idade = 20;
    printf( A idade do aluno eh: %d anos.. , idade);
```

```

cout << endl;
system("PAUSE");
return 0;
}

```

Questão 03. Operadores de Atribuição Composta e Avaliação Sequencial (Cap. 2) — Os operadores de atribuição em C executam suas ações da direita para a esquerda e podem ser combinados com operadores aritméticos. Determine os valores finais de a, b, c e d após a execução da sequência abaixo:

```

int a = 2, b = 4, c = 5, d = 10;
a += b + c;           // Valor final de a = ?
b *= c - 2;           // Valores finais de b e c = ?
d %= a + 3;           // Valor final de d = ?
a += b += c += 5;     // Valores finais de a, b e c = ?

```

Questão 04. Avaliação de Expressões Lógicas, Relacionais e Precedência (Cap. 2) — Considere as variáveis inteiras i = 2, j = 3, k = 0 e as variáveis de ponto flutuante x = 2.5, y = 5.0. Avalie cada expressão abaixo e determine seu resultado lógico em C (1 para Verdadeiro, 0 para Falso):

- i < j + 2 => Resultado: ?
- 2 * i - 5 <= j - 4 => Resultado: ?
- !k && (x + y >= 7.5) => Resultado: ?
- !(i == j) || (y / x == 2.0) => Resultado: ?
- i == 2 && j == 4 || k == 0 => Resultado: ?

Questão 05. Estruturas de Repetição: Comparação entre for, while e do-while (Cap. 3) — As estruturas de repetição permitem a execução iterativa de instruções em C. Analise as características de for, while e do-while e responda fundamentadamente:

- Qual é a diferença essencial entre while e do-while em relação ao número mínimo de execuções do bloco de código e ao momento do teste condicional?
- Em que cenários o laço for se apresenta como a escolha mais elegante e legível frente ao laço while?
- O trecho de código 'while (condicao);' (com ponto-e-vírgula ao final do cabeçalho) constitui um erro de compilação ou de lógica? O que acontece se condicao for verdadeira?

Questão 06. Escopo de Bloco e Comandos de Desvio (break e continue) (Cap. 3) — Analise o programa abaixo que calcula a soma acumulada de quadrados dentro de um laço for contendo um comando de desvio e controle de escopo interno:

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;
    for (i = 1; i <= 10; i++) {
        if (i == 5) continue;
        if (i == 8) break;
        int soma = 0;
        soma += i * i;
    }
    printf("Soma final = %d\n", soma);
    system("PAUSE");
}

```

```
    return 0;
}
```

- a) Por que o compilador emitirá um erro de compilação na instrução `printf` final?
- b) Quais iterações do laço serão efetivamente executadas e qual o impacto dos comandos `continue` e `break` no fluxo?
- c) Reescreva o código corrigindo o escopo de 'soma' e apresente o resultado que será impresso no console.

PARTE II: QUESTÕES PRÁTICAS DE IMPLEMENTAÇÃO (CÓDIGO FONTE EM C)

Questão 8. Cálculos Geométricos e Constantes com `<math.h>` — Desenvolva um programa em C que solicite ao usuário o valor do raio R de uma esfera. Defina a constante PI como 3.14159265 e calcule: a) A área da superfície da esfera ($A = 4 * PI * R^2$); b) O volume da esfera ($V = (4.0/3.0) * PI * R^3$). Utilize a função `pow()` da biblioteca `<math.h>` e exiba os resultados formatados com 3 casas decimais. Atenção para a divisão real de 4.0 por 3.0!

Questão 9. Geometria do Triângulo e Fórmula de Heron — Escreva um programa em C que leia os comprimentos dos três lados (a , b , c) de um triângulo qualquer. Sabendo que o semiperímetro p é dado por $(a + b + c) / 2.0$, calcule a área do triângulo utilizando a **Fórmula de Heron**: $Area = \sqrt{p * (p - a) * (p - b) * (p - c)}$. Utilize a função `sqrt()` da biblioteca `<math.h>`.

Questão 10. Resto da Divisão (%) e Decomposição do Tempo — Desenvolva um programa em C que receba uma quantidade inteira de segundos informada pelo usuário. O programa deve calcular e exibir o tempo equivalente decomposto em Horas, Minutos e Segundos restantes (Exemplo: 3665 segundos correspondem a 1 hora, 1 minuto e 5 segundos).

Questão 11. Cálculo Salarial com Gratificação e Impostos — Uma empresa contrata um técnico a R\$ 45,00 por dia trabalhado. Crie um programa em C que solicite o número de dias trabalhados, calcule o salário bruto, adicione uma gratificação de 5% sobre o bruto e desconte 8% de imposto de renda sobre o bruto. Ao final, exiba o holerite detalhado com o valor líquido a receber.

Questão 12. Validação de Entrada de Dados com Laço Garantido ('do-while') — Escreva um programa em C que solicite ao usuário uma nota válida no intervalo fechado de 0.0 a 10.0. Caso o usuário digite um valor inválido (como -2.5 ou 11.0), o programa deve exibir uma mensagem de erro e repetir a solicitação utilizando a estrutura `'do-while'`. O programa só deve encerrar quando uma nota válida for digitada.

Questão 13. Cálculo do Fatorial com Tratamento do Zero e Tipo `'long long int'` — Escreva um programa em C que solicite um número inteiro N e calcule o seu fatorial ($N!$). Lembre-se que $0! = 1$ e $1! = 1$. O programa deve utilizar a variável do resultado como `'long long int'` com o especificador `'%lld'` para evitar estouro de memória e tratar entradas inválidas (números negativos).

Questão 14. Autenticação de Senha com Limite Finito de Tentativas — Desenvolva um sistema de autenticação que defina uma senha numérica secreta (ex: 2026). O programa deve permitir que o usuário tente digitar a senha no máximo 3 vezes usando um laço `'while'` ou `'for'`. Se acertar, exiba 'Acesso Concedido!' e encerre; se errar as 3 vezes, exiba 'Conta Bloqueada por Segurança!'.

Questão 15. Geração de Padrões Visuais com Laços Aninhados: Triângulo de Floyd — Escreva um programa em C que leia um número inteiro positivo N e imprima N linhas do ****Triângulo de Floyd**** utilizando laços aninhados. Por exemplo, para N = 5, a saída no console deve ser exatamente:

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Fim da Lista de Exercícios • Cesar School 2026.2
Desejamos excelente estudo e prática de programação!